In [2]:
```
#1 Import libraries and load datasets

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import confusion_matrix, classification_report

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense

train_df = pd.read_csv("sign_mnist_train.csv")
test_df = pd.read_csv("sign_mnist_test.csv")
```

In [3]:
```
#2 Separate features and labels

X_train = train_df.drop("label", axis=1).values
y_train = train_df["label"].values

X_test = test_df.drop("label", axis=1).values
y_test = test_df["label"].values

# Select one class as anomaly (example: label 24)
anomaly_class = 24

# Convert pixel values to range 0-1
X_train = X_train / 255.0
X_test = X_test / 255.0

# Train only on the remaining 23 classes
X_normal = X_train[y_train != anomaly_class]
```

In [4]:
```
#3 Build and train Autoencoder

model = Sequential([
    Input(shape=(784,)),
    Dense(128, activation="relu"),
```

```
    Dense(64, activation="relu"),
    Dense(128, activation="relu"),
    Dense(784, activation="sigmoid")
])

model.compile(optimizer="adam", loss="mse")

model.fit(
    X_normal, X_normal,
    epochs=10,
    batch_size=128,
    validation_split=0.1
)
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are ins
talled, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/10
186/186 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - loss: 0.0241 - val_loss: 0.0174
Epoch 2/10
186/186 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0140 - val_loss: 0.0117
Epoch 3/10
186/186 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0108 - val_loss: 0.0097
Epoch 4/10
186/186 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0093 - val_loss: 0.0091
Epoch 5/10
186/186 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0087 - val_loss: 0.0088
Epoch 6/10
186/186 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0081 - val_loss: 0.0079
Epoch 7/10
186/186 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0077 - val_loss: 0.0076
Epoch 8/10
186/186 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0074 - val_loss: 0.0073
Epoch 9/10
186/186 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0072 - val_loss: 0.0074
Epoch 10/10
186/186 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.0070 - val_loss: 0.0069
```

Out[4]: <keras.src.callbacks.history.History at 0x1a260f46f90>

In [5]:
```
#4 Calculate reconstruction error and threshold
```

```
train_pred = model.predict(X_normal, verbose=0)

train_error = np.mean(
    np.square(X_normal - train_pred),
    axis=1
)

threshold = train_error.mean() + 3 * train_error.std()

print("Anomaly Threshold:", threshold)
```

```
Anomaly Threshold: 0.015782737498941855
```

In [6]:

```
#5 Detect anomalies in test data

test_pred = model.predict(X_test, verbose=0)

test_error = np.mean(
    np.square(X_test - test_pred),
    axis=1
)

# 1 = anomaly, 0 = normal
y_pred = (test_error > threshold).astype(int)

# Actual labels: 1 if selected class, otherwise 0
y_true = (y_test == anomaly_class).astype(int)

print("Confusion Matrix:")
print(confusion_matrix(y_true, y_pred))

print("Classification Report:")
print(classification_report(y_true, y_pred, zero_division=0))
```

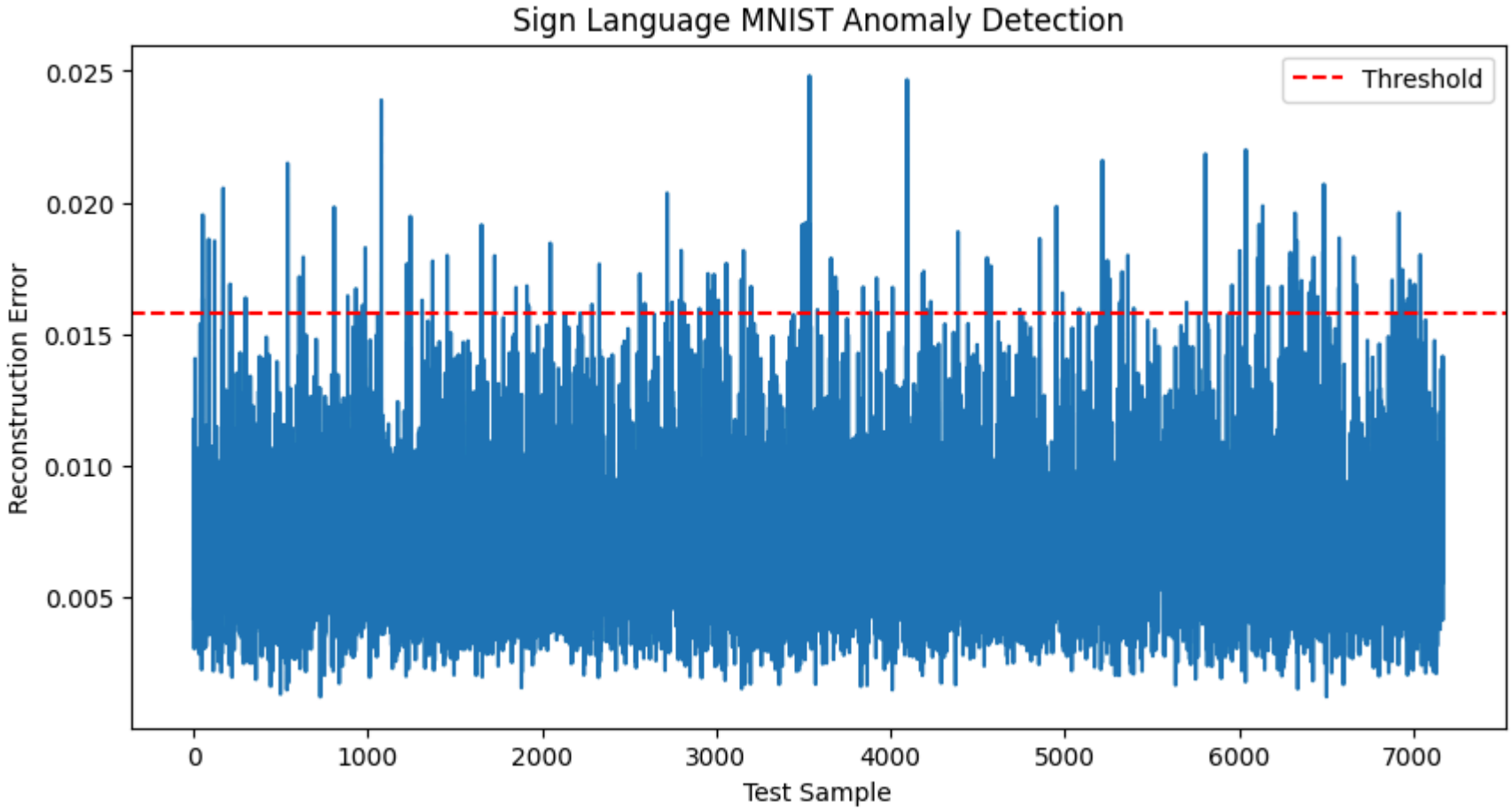
```
Confusion Matrix:
[[6726  114]
 [ 314   18]]
Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.98      0.97      6840
           1       0.14      0.05      0.08       332

    accuracy                           0.94      7172
   macro avg       0.55      0.52      0.52      7172
weighted avg       0.92      0.94      0.93      7172
```

In [7]:
```python
#6 Plot reconstruction error

plt.figure(figsize=(10, 5))
plt.plot(test_error)
plt.axhline(threshold, color="red", linestyle="--", label="Threshold")
plt.xlabel("Test Sample")
plt.ylabel("Reconstruction Error")
plt.title("Sign Language MNIST Anomaly Detection")
plt.legend()
plt.show()
```


Sign Language MNIST Anomaly Detection
Threshold
Reconstruction Error
0.025
0.020
0.015
0.010
0.005
0
1000
2000
3000
4000
5000
6000
7000
Test Sample

In [ ]: